

DETECT LABELS WITH AMAZON REKOGNITION



INTRODUCTION & OBJECTIVES

Introduction

- Amazon Rekognition is a cloud-based image analysis service by AWS.
- It uses deep learning to detect and label objects, people, text, scenes, and activities within images.

Primary Objective

- To automatically detect and label objects in an image using Amazon Rekognition API.
- This project demonstrate how AWS services can be used for intelligent image analysis.
- Build scalable cloud-based solution

AWS SERVICES USED



Amazon Rekognition

- Detects objects and labels



Amazon S3

- Stores input and Output images



IAM

- Manages access permissions



Flask

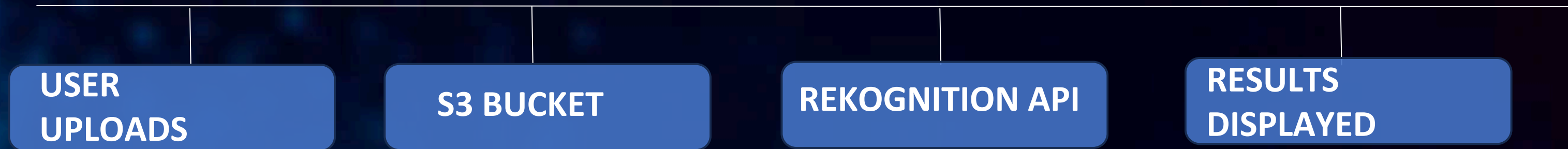
- Provides the web interface



Boot3

- Python SDK to communicate with AWS services

SYSTEM ARCHITECTURE



1. User uploads image through Flask web application interface with intuitive drag-and-drop functionality.
2. Image gets stored in Amazon S3 bucket with proper access permissions and versioning enabled.
3. AWS Rekognition processes image using `detect_labels()` API call with configurable confidence thresholds.
4. Processed results with bounding boxes and confidence scores displayed on web interface in JSON format.

IMPLEMENTATION STEPS

1. Create and configure S3 bucket.
2. Set up IAM user and policies.
3. Write Flask + Boto3 code to connect with Rekognition.
4. Upload image and display detection results.
5. Test application with various images.

AMAZON S3 SETUP

- Create bucket: *rekognition-label-bucket*
- Region: *ap-south-1 (Mumbai)*
- Folders: /uploads for input, /result for output

UPLOAD FROM FLASK

```
s3_client.upload_file(local_file, 'rekognition-label-bucket', 'uploads/' + filename)
```

- Flask uploads image to S3 /uploads

REKOGNITION ACCESS

```
Image={'S3Object': {'Bucket': 'rekognition-label-bucket', 'Name': 'uploads/' + filename}}
```

- Rekognition fetches image directly from S3

FLOW DIAGRAM

Flask Upload → S3 Bucket → Rekognition → S3 Result → Flask Display



IAM CONFIGURATION



Create an IAM User

1. Go to AWS Console → IAM → Users → Add User
2. Choose Programmatic access (for API usage through Python)
3. Click Next: Permissions

Attach Required Policies

Attach these managed AWS policies:

- AmazonS3FullAccess → allows reading/writing images in S3
- AmazonRekognitionFullAccess → allows label detection through Rekognition

Generate Access Keys

After creating the IAM user:

- Download the .csv file that contains:
 - Access Key ID
 - Secret Access Key

Integrate with Python (Boto3)

Use the IAM credentials in your Python code:

- These credentials let the Flask app connect securely to AWS services.

python

```
import boto3
```

```
rekognition = boto3.client(  
    'rekognition',  
    aws_access_key_id='YOUR_ACCESS_KEY',  
    aws_secret_access_key='YOUR_SECRET_KEY',  
    region_name='ap-south-1'  
)
```

```
s3 = boto3.client(  
    's3',  
    aws_access_key_id='YOUR_ACCESS_KEY',  
    aws_secret_access_key='YOUR_SECRET_KEY',  
    region_name='ap-south-1'
```


CONNECT FLASK APP TO S3 USING BOTO3

Boto3 connects our Python app to AWS services programmatically

**Initialize
AWS Clients**

Here we create S3 and Rekognition clients using Boto3.

```
import boto3

s3 = boto3.client(
    's3',
    aws_access_key_id='YOUR_ACCESS_KEY',
    aws_secret_access_key='YOUR_SECRET_KEY',
    region_name='ap-south-1'
)

rekognition = boto3.client(
    'rekognition',
    aws_access_key_id='YOUR_ACCESS_KEY',
    aws_secret_access_key='YOUR_SECRET_KEY',
    region_name='ap-south-1'
)
```

**Upload
Image to S3**

When a user uploads an image through Flask:
This sends the image from your local folder to the S3 bucket.

```
s3.upload_file('static/input.jpg', 'rekognition-label-bucket', 'uploads/in
```

**Detect Labels
using
Rekognition**

Use the S3 image path for label detection:
This returns labels (like *Car, Person, Road*) with confidence percentages.

```
response = rekognition.detect_labels(
    Image={'S3Object': {'Bucket': 'rekognition-label-bucket', 'Name': 'uploads/input.jpg'}},
    MaxLabels=5
)
```

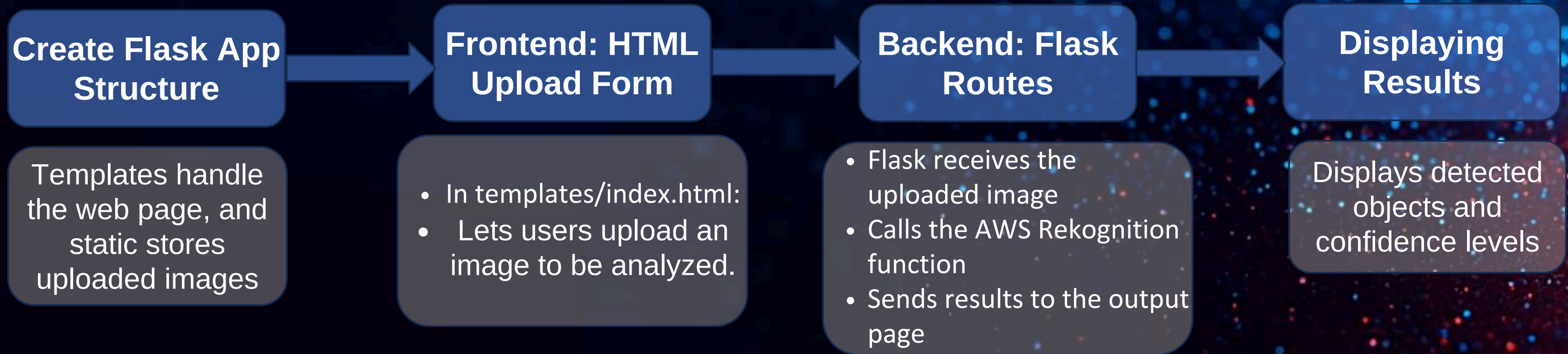
**Display
Results**

Extract and display labels:
Then show results on the Flask web page.

```
for label in response['Labels']:
    print(label['Name'], label['Confidence'])
```


INTEGRATE FLASK FRONTEND

Flask acts as the bridge between the user and the AWS backend



```
bash

/static      → stores images (input/output)
/templates   → contains HTML files
app.py       → main Flask application
```

```
html

<form action="/detect" method="POST" enctype="multipart/form-data">
  <input type="file" name="file">
  <button type="submit">Detect Labels</button>
</form>
```

```
python

@app.route('/detect', methods=['POST'])
def detect():
    file = request.files['file']
    file.save('static/input.jpg')
    # Upload to S3 and call Rekognition here
    return render_template('result.html', labels=labels)
```

```
html

<h3>Detected Labels:</h3>
<ul>
  {% for label in labels %}
    <li>{{ label.Name }} - {{ label.Confidence }}%</li>
  {% endfor %}
</ul>
```


DISPLAY DETECTION RESULTS

This is the final stage where the system shows what Amazon Rekognition detected

Retrieve Rekognition Output

Rekognition returns structured data — label names and confidence percentages

Format the Result

We format the JSON data into readable text for the web page

Visualize on Flask Web Page

Each detected object and its confidence value are shown dynamically on the page

python

```
response = rekognition_client.detect_labels(  
    Image={'Bytes': source_bytes},  
    MaxLabels=max_labels,  
    MinConfidence=80  
)
```

python

```
for label_info in response['Labels']:  
    res_response[key] = {  
        'Name': label_info['Name'],  
        'Confidence': label_info['Confidence']  
    }
```

html

```
<div class="result-box">  
    <code>  
        {% for key, val in result_json_response.items() %}  
            {{ val['Name'] }} : <kbd>{{ val['Confidence'] }}%</kbd><br>  
        {% endfor %}  
    </code>  
</div>
```


RESULTS

Image 1 : Motorcycle, Bicycle, Person

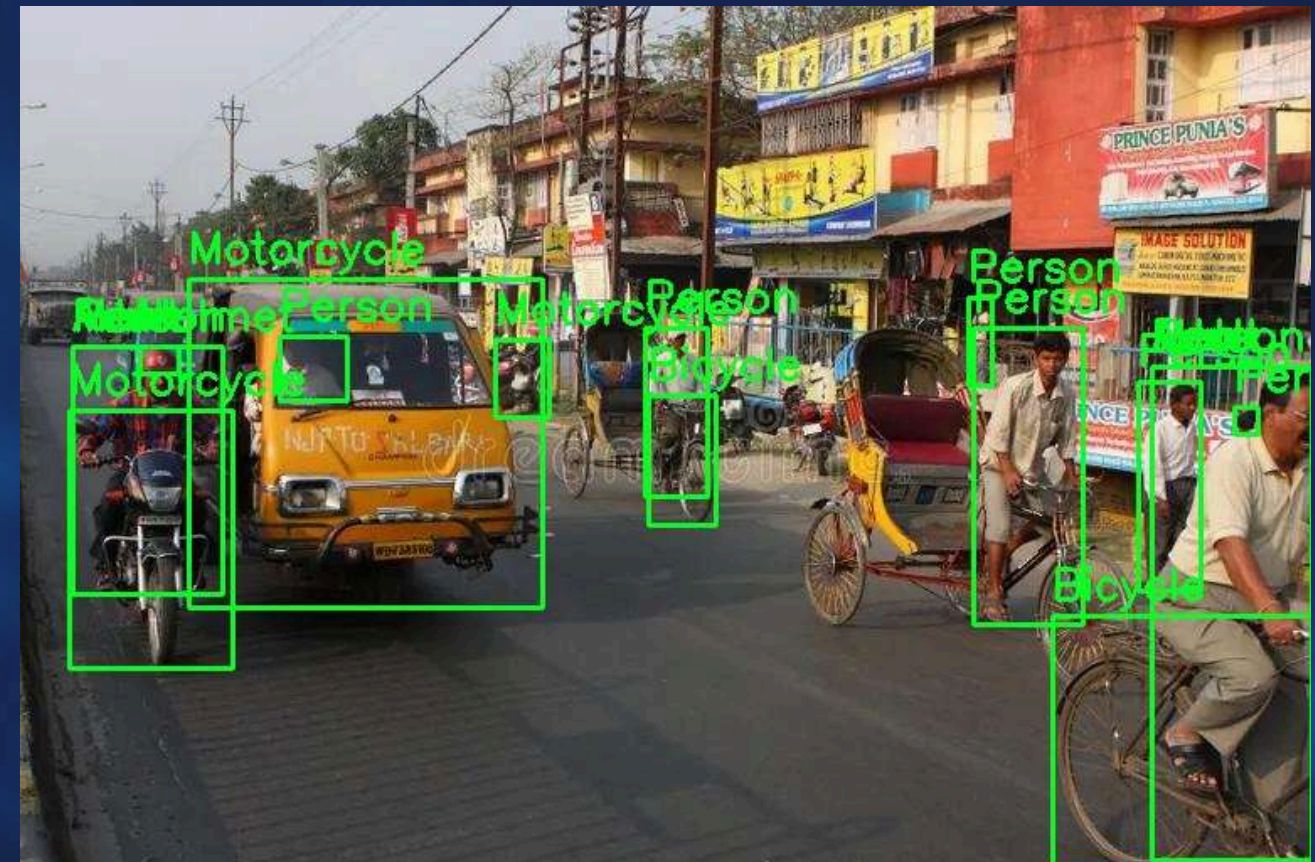
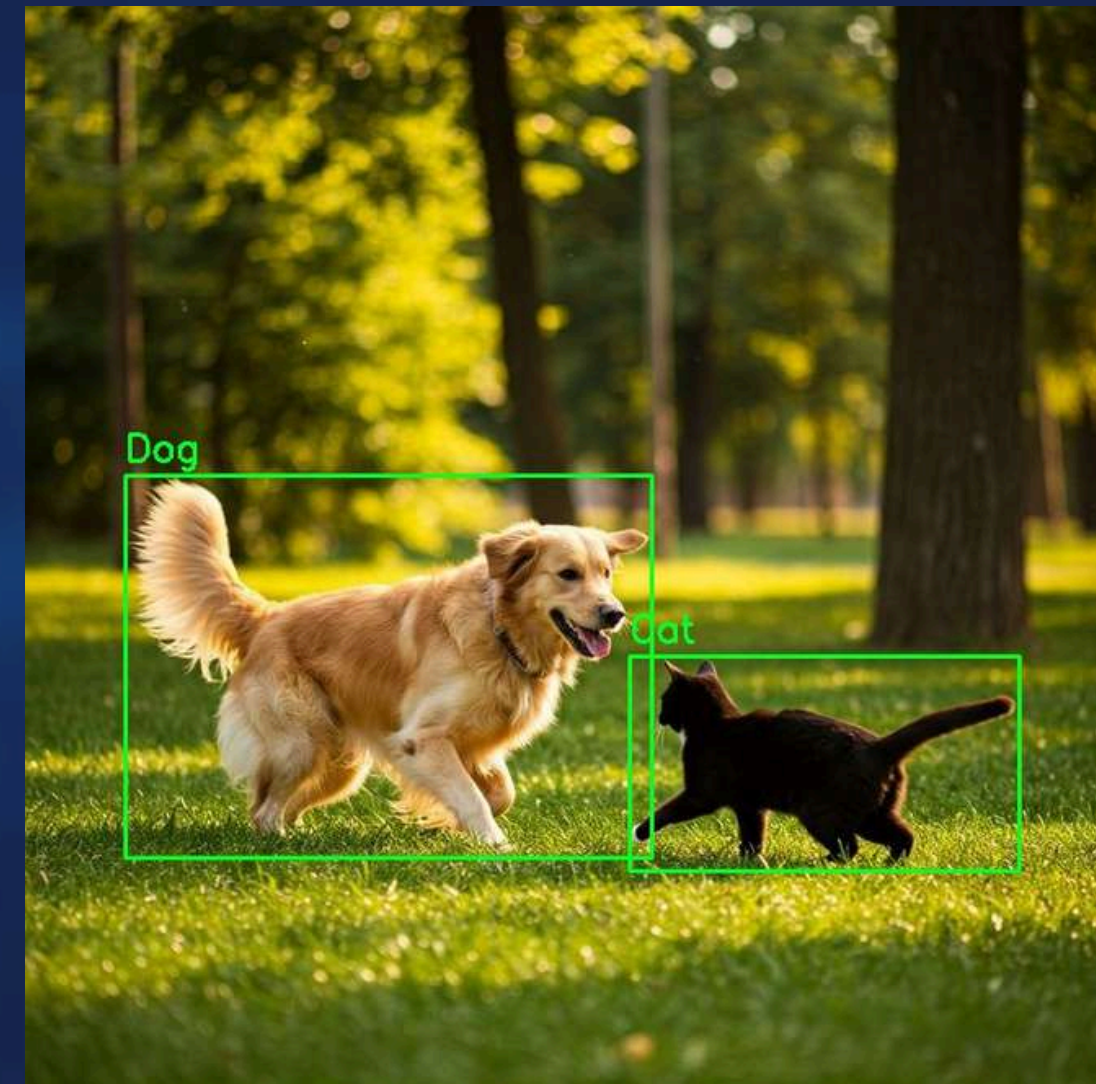
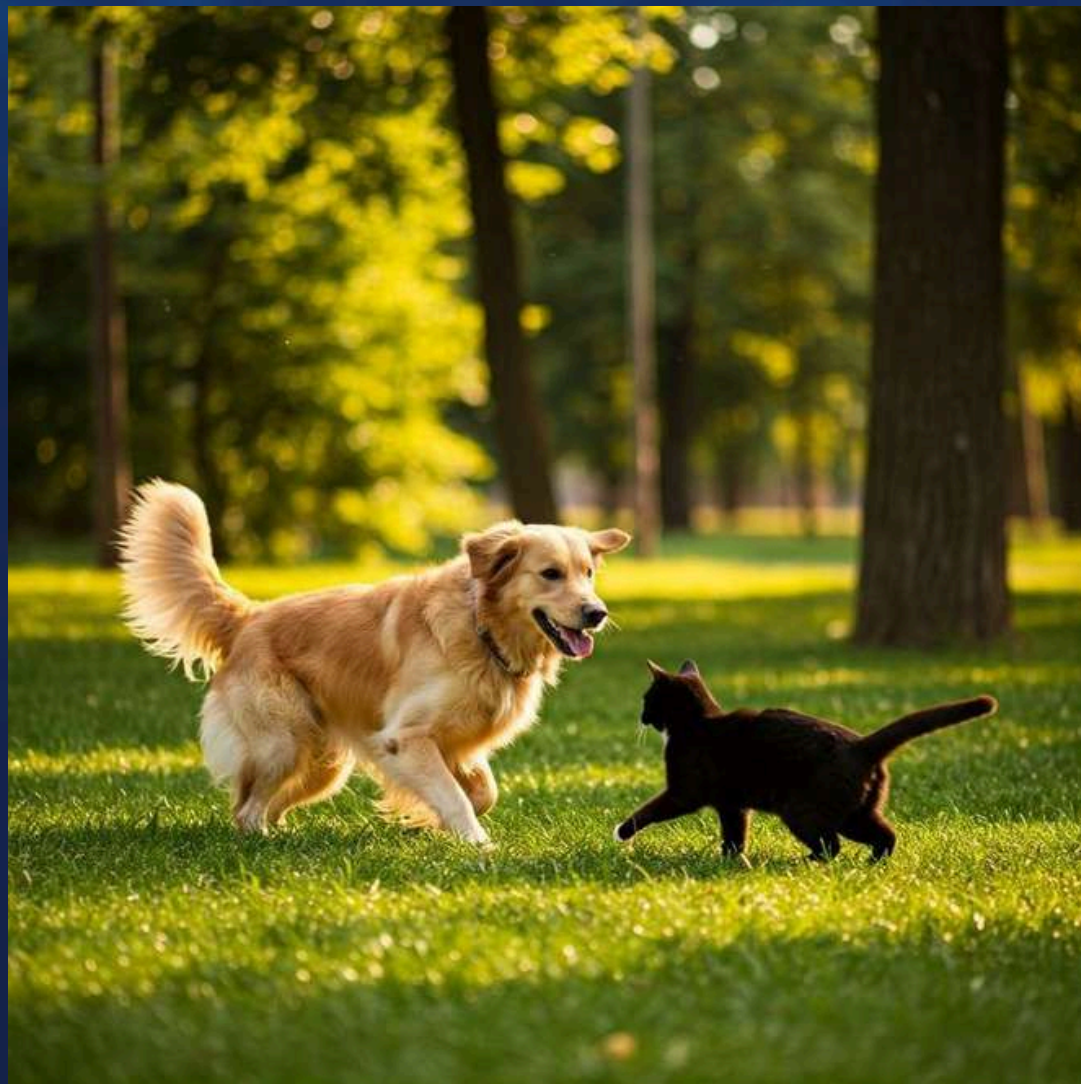


Image 2 : Dog and Cat



CONCLUSION

- **The project successfully demonstrates image label detection using AWS Rekognition.**
- **It showcases cloud-based AI capabilities and real-world applicability.**





THANK YOU